

Beating the Locals: A Compact Convolutional Network for Street-Level Geolocation in Southern California

Joseph Calderon

DSC 140B, University of California, San Diego

March 2026

Abstract

GeoGuessr, an online game that asks players to identify where a single Google Street View image was taken, has revealed a surprising human capacity for visual geolocation. This work asks how much of that ability a small model can recover on a narrower version of the task: predicting which of six Southern California cities a Street View image was taken in, namely Anaheim, Bakersfield, Los Angeles, Riverside, San Luis Obispo (SLO), and San Diego. To gauge the difficulty, we first collected a human baseline on a fifty-image sample, on which the author scored 24% and a twenty-year Los Angeles resident scored 40%. We then trained a compact convolutional neural network with only 33,510 parameters on 9,181 images, which reached 83.6% validation accuracy and comfortably surpassed both. An analysis of the errors shows that they align with geographic intuition: the model mostly confuses the visually similar, sprawling cities in and around Los Angeles, while more distinctive locations such as Riverside and SLO are rarely missed.

1 Introduction

In recent years, an internet game has become a viral sensation for its difficulty and the seemingly inhuman abilities of those who are proficient at it. This game, known as GeoGuessr, picks a random spot on Earth and presents the player with an image of that spot from Google Street View. The goal of the player is to determine the location where the picture was taken. Some players are so skilled that they can pinpoint locations anywhere on Earth from subtle differences in infrastructure such as stoplights, shades of the sky, and even specific species of grass (see: YouTube user Rainbolt guessing locations of empty grass fields).

The goal of this project is to build a framework for a model that can compete with humans at GeoGuessr. For simplicity and reproducibility, we narrow the problem to a small dataset of 9,181 images taken throughout Southern California, and design the model to predict which of six cities (Los Angeles, San Diego, SLO, Bakersfield, Riverside, or Anaheim) a picture was taken in. The narrower setting keeps the dataset manageable and the experiment easy to reproduce, but it does not make the task easy: all six cities lie in the same region and share a climate, vegetation, and road infrastructure, which makes them genuinely easy to confuse.

To that end, we first measure how well people perform at the task, since a model is only compelling if it can outperform a reasonable human. We then describe the network, detail the training procedure, and examine the kinds of mistakes the model makes and why.

2 Related Work

None of the machinery used here is novel, so it is worth placing the project in context. Convolutional neural networks have been the standard tool for image classification for well over a decade [1, 2]. The two components this work relies on most, batch normalization and the Adam optimizer, are now standard practice: batch normalization keeps intermediate activations well-behaved and accelerates training [3], while Adam adapts the step size of each parameter and tends to converge in fewer iterations than plain stochastic gradient descent [4].

Treating geolocation as a classification problem is also well established. Weyand et al. built PlaNet, which partitions the globe into a grid of cells and predicts the cell to which an image belongs, and showed that a CNN can outperform well-traveled humans at the worldwide version of the game [5]. The present study is far smaller in scope, but it shares that framing, and the same comparison against human players.

3 Dataset

The dataset comprises 9,181 Google Street View images, each labeled with the city in which it was taken. The classes are close to balanced, ranging from 1,429 images for Los Angeles to 1,588 for SLO, so plain accuracy serves as a meaningful metric and no resampling is required. Files follow the naming pattern `City-hash.jpg`, so the label is recovered by reading the substring preceding the final hyphen.

Before training, 20% of the data is held out as a validation set using a fixed random seed, leaving 1836 validation images and 7,345 training images. The same seeded split is used for training, model selection, and final evaluation, so all reported numbers are measured on the same held-out images and remain comparable across runs.

Each image is then resized to a 64 by 32 pixel raster (height by width) and converted to a three-channel tensor with values in $[0, 1]$. This is an aggressive reduction: the model sees only $64 \times 32 \times 3 = 6,144$ values per image. Such a low resolution discards fine detail such as legible text on signage, precisely the kind of cue a skilled human exploits, so the resulting accuracy is best read as a floor for this architecture rather than a ceiling.

4 Human Baseline

To gauge the difficulty of the task, we first measured human performance on a sample of fifty images. The author, labeling the sample by hand, achieved an accuracy of 24% (Figure 1, left). As a stronger baseline, a roommate who has lived in Los Angeles for twenty years labeled the same images and achieved 40% (Figure 1, right). Both scores sit well above the 16.7% expected from random guessing, yet neither is close to reliable.

The confusion matrices in Figure 1 reveal where the errors fall, and the pattern is telling. Neither participant could cleanly separate the inland Los Angeles-area cities, and both defaulted to Los Angeles when uncertain. The local resident was markedly better on Los Angeles itself, correctly identifying seven of nine, which is intuitive: familiarity helps with recognizing one’s own city but offers little advantage for its neighbors. For the record, the author is from Northern California.

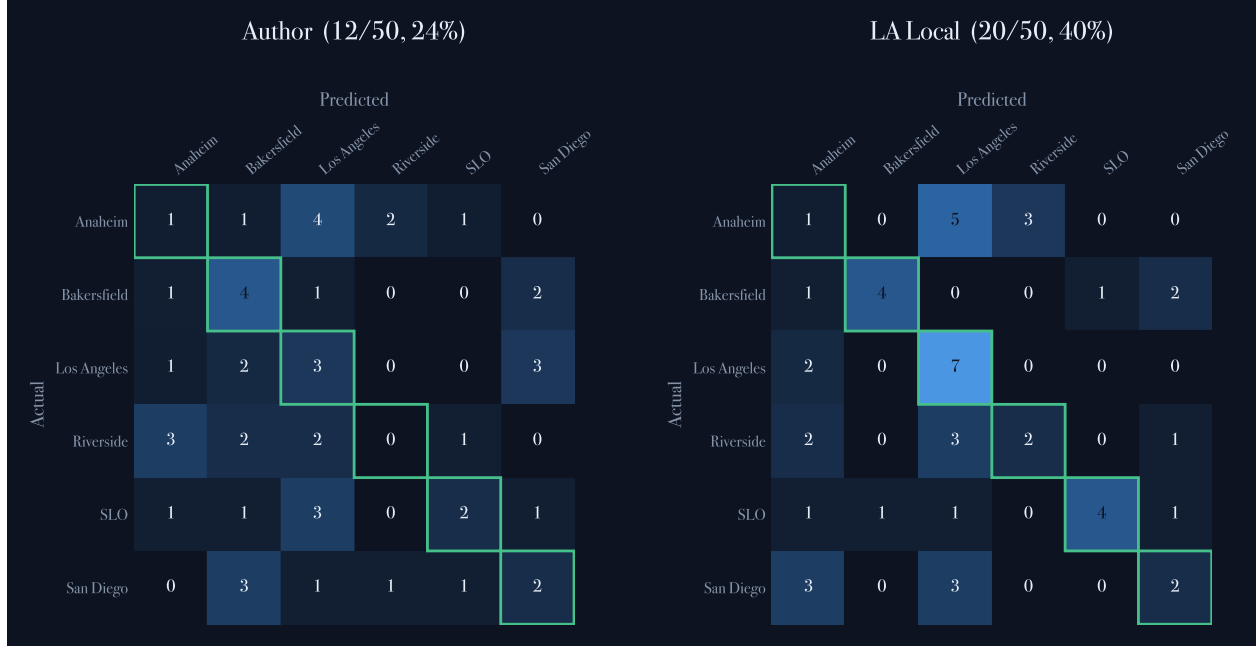


Figure 1: Human baseline confusion matrices for the author (left, 24%) and a twenty-year Los Angeles resident (right, 40%). Rows are the true city and columns the guess; cells outlined in green lie on the diagonal and count correct answers. Color encodes the row-normalized rate. Both participants confuse the adjacent Los Angeles-basin cities and default to “Los Angeles” when uncertain.

5 Model Architecture

This task calls for a convolutional neural network (CNN), largely because it is a multi-class classification problem on RGB images, exactly the setting for which CNNs are best suited. The model consists of four convolutional blocks with 16, 32, 32, and 64 filters, respectively. Each block applies a 3×3 filter with unit padding, followed by batch normalization and a ReLU activation; the first block additionally applies a max pool to the resulting response map. After the four blocks, global average pooling collapses the final feature map to a single 64-dimensional vector, which keeps the classifier compact and provides a degree of translation invariance. The output layer carries no activation, since training uses `nn.CrossEntropyLoss()`, which operates on raw logits.

Listing 1: The model definition. The output layer produces one logit per city.

```
model = nn.Sequential(
    nn.Conv2d(3, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
    nn.Conv2d(32, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
    nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
    nn.AdaptiveAvgPool2d((1, 1)),
    nn.Flatten(),
    nn.Linear(64, 6),
)
```

Parameter count. The number of learnable parameters in convolutional block i follows from the filter volume, the number of filters, the bias terms, and the parameters of `nn.BatchNorm2d()`:

$$\text{params}_i = (n_{i-1} \cdot k \times k \cdot n_i) + n_i + 2 \cdot n_i = n_i (n_{i-1} k^2 + 3), \quad (1)$$

where k is the filter height and width and n_i is the number of filters in block i . The first term counts the weights across all convolution filters, the second counts the per-filter biases, and the third, $2 \cdot n_i$, accounts for the scale and shift that batch normalization learns per filter. With $n_0 = 3$ for the three channels of an RGB image, the four blocks contribute

$$16(3 \cdot 3^2 + 3) + 32(16 \cdot 3^2 + 3) + 32(32 \cdot 3^2 + 3) + 64(32 \cdot 3^2 + 3) = 33,120, \quad (2)$$

and the output layer adds $6 \cdot (64 + 1) = 390$, for a total of 33,510. Table 1 lists the architecture block by block.

Table 1: Architecture summary for a $3 \times 64 \times 32$ input. Output shapes are given as channels $\times$ height $\times$ width.

Block	Output shape	Parameters
Conv(3→16), BN, ReLU, MaxPool	$16 \times 32 \times 16$	480
Conv(16→32), BN, ReLU	$32 \times 32 \times 16$	4,704
Conv(32→32), BN, ReLU	$32 \times 32 \times 16$	9,312
Conv(32→64), BN, ReLU	$64 \times 32 \times 16$	18,624
Global average pool	$64 \times 1 \times 1$	0
Linear(64→6)	6	390
Total		33,510

6 Training Procedure

Training uses the Adam optimizer with a learning rate of 0.0003. This choice is largely because Adam can reach good weights in fewer iterations than SGD in some cases, which keeps training time low and aids reproducibility; the full run takes roughly fifteen minutes on a CPU for the 9,181 images. Because the task is a mutually exclusive classification problem, the objective is `nn.CrossEntropyLoss()`.

Reproducibility was a particular concern. Python, NumPy, and PyTorch are seeded at the start of training, which fixes the weight initialization, the batch ordering, and the train/validation split, so a fresh clone of the project reproduces the reported numbers.

Checkpoint selection. The model is trained for 80 epochs, with validation accuracy recorded after each. Rather than keeping the single highest-scoring epoch, which on a small validation set is easily a lucky spike, we adopt a more conservative rule: keep the earliest epoch whose accuracy falls within 1% of the best value observed across the run. This selects a checkpoint that is statistically indistinguishable from the peak but trained for fewer epochs, which reduces the risk of overfitting. For this run, the rule selects epoch 45.

Figure 2 shows the training dynamics, which are promising. Training accuracy climbs steadily to around 91%, while validation accuracy rises sharply over the first twenty epochs and then settles into a noisy band in the low-to-mid eighties. The modest gap between the curves is the expected

sign of mild overfitting, and its small size suggests that the compact model and global average pooling supply enough regularization on their own, without data augmentation. The noise in the validation curve, which still swings by several points late in training, largely reflects the limited size of the validation set.

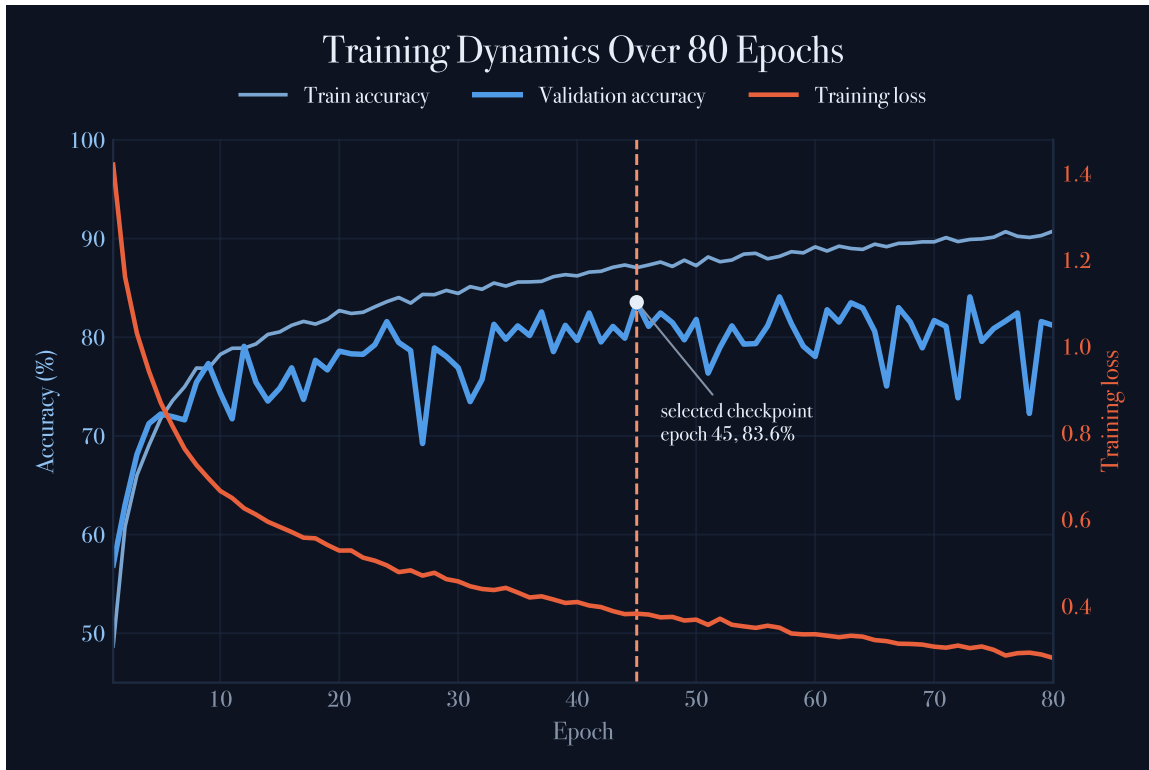


Figure 2: Training dynamics over 80 epochs. Validation accuracy (dark blue, left axis) rises sharply and then plateaus, training accuracy (light blue, left axis) keeps climbing, and the training loss (coral, right axis) falls smoothly. The dashed line marks the selected checkpoint at epoch 45.

7 Results

The selected model attains an overall validation accuracy of 83.6%, correctly classifying 1534 of 1836 held-out images. This far exceeds both human baselines, the author’s 24% and the resident’s 40%, as well as the 16.7% random-chance rate; Figure 3 places the three side by side.

Per-class performance. Performance is far from uniform across the six cities. Figure 4 ranks them by recall, and Table 2 reports precision, recall, and F1 score for each. Riverside leads by a wide margin at 94.0% recall, with SLO, San Diego, and Bakersfield following between roughly 83% and 88%. Anaheim and Los Angeles are the two clear laggards, each near 75%.

Error structure. The confusion matrix in Figure 5 shows where the remaining errors concentrate. The largest off-diagonal entries are the mutual confusions between Bakersfield and Anaheim (32 and 31 images) and between Anaheim and Los Angeles (29 in each direction). Almost all of the heavy confusion therefore involves Anaheim, Bakersfield, and Los Angeles, a set of sprawling, suburban cities that share dense arterial roads, palm trees, and similar signage, leaving little to

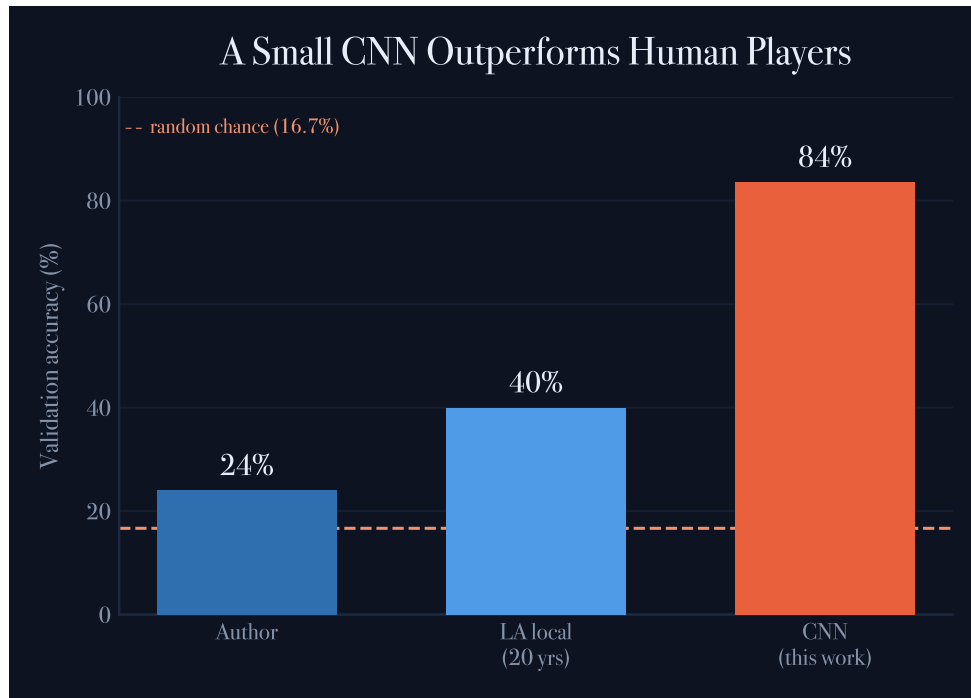


Figure 3: Validation accuracy of the two human players and the CNN. The model more than doubles the strongest human score, and reaches it without the lifetime of residence that the strongest human baseline required.

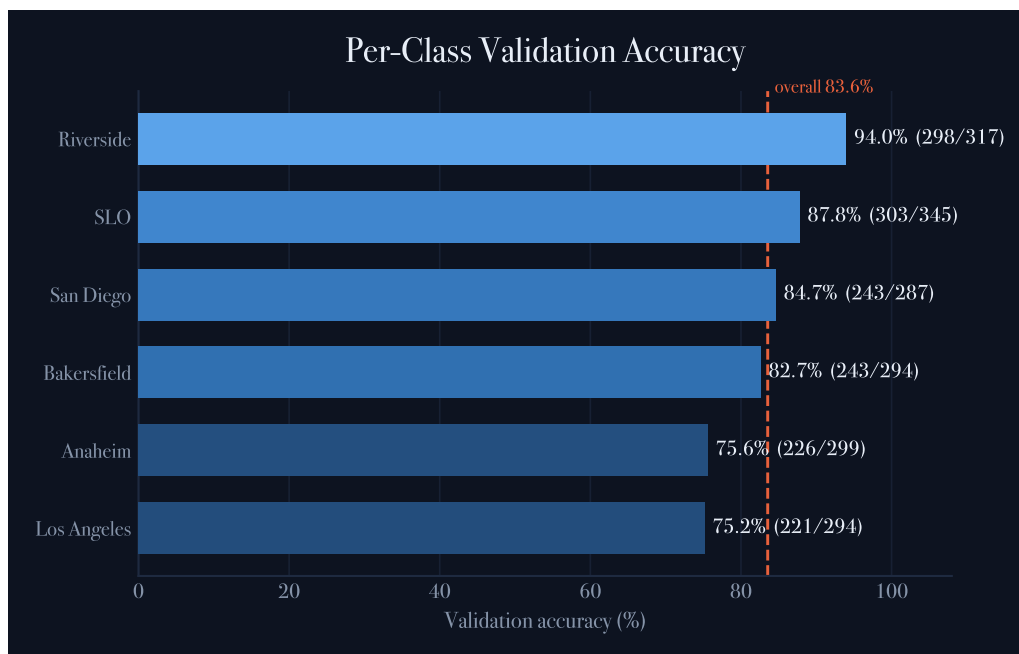


Figure 4: Per-class validation accuracy (recall), sorted from best to worst. The dashed line marks the overall accuracy of 83.6%. The roughly nineteen-point spread between the best and worst city is a direct reflection of the region’s geography.

Table 2: Per-class precision, recall, and F1 score (in percent) on the validation set, with support (the number of validation images per class). All values are computed from the confusion matrix in Figure 5.

City	Precision	Recall	F1	Support
Anaheim	74.6	75.6	75.1	299
Bakersfield	84.1	82.7	83.4	294
Los Angeles	71.1	75.2	73.1	294
Riverside	89.2	94.0	91.6	317
SLO	93.5	87.8	90.6	345
San Diego	88.4	84.7	86.5	287
Overall	83.6% accuracy			1836

distinguish them at 64 by 32 pixels. Riverside is the cleanest class by a wide margin and is rarely confused with anything: its largest off-diagonal entry is just seven images. SLO is close behind, its only notable slip being the 25 images read as Los Angeles. Los Angeles itself acts as a sink, drawing 90 misclassifications from the other five cities combined. This mirrors the human baselines, where both participants also fell back on Los Angeles when uncertain; the model simply makes far fewer such mistakes.

8 Limitations

Several limitations qualify these results. First, the 64 by 32 input discards considerable detail, including legible signage, so accuracy would likely improve at a higher resolution; the figures reported here are best read as a floor for this architecture. Second, evaluation uses a single validation split rather than a separate test set or cross-validation, and because that split also guided checkpoint selection, the reported accuracy probably overstates performance on truly unseen data slightly. Third, the problem is deliberately narrow, six cities in a single region, and the approach has not been tested with more classes or a wider geographic range. Finally, the human baseline rests on fifty images labeled by two participants, and is therefore better regarded as a sanity check than a precise measurement.

9 Conclusion

This project trained a compact convolutional neural network to predict which of six Southern California cities a Google Street View image was taken in. With only 33,510 parameters and a heavily downsampled input, the model reached 83.6% validation accuracy, more than doubling the score of a twenty-year Los Angeles resident and far exceeding both the out-of-region and random-chance baselines. Its errors mirror those of the human players: the model mostly confuses the sprawling cities in and around Los Angeles and rarely misses the more distinctive ones.

These results suggest that even a very small model can capture regional cues that people find difficult to articulate. Natural next steps include training at a higher resolution to recover signage and fine texture, applying data augmentation or a deeper pretrained backbone to close the remaining gap, and broadening the task to additional cities and regions to test how the approach scales.

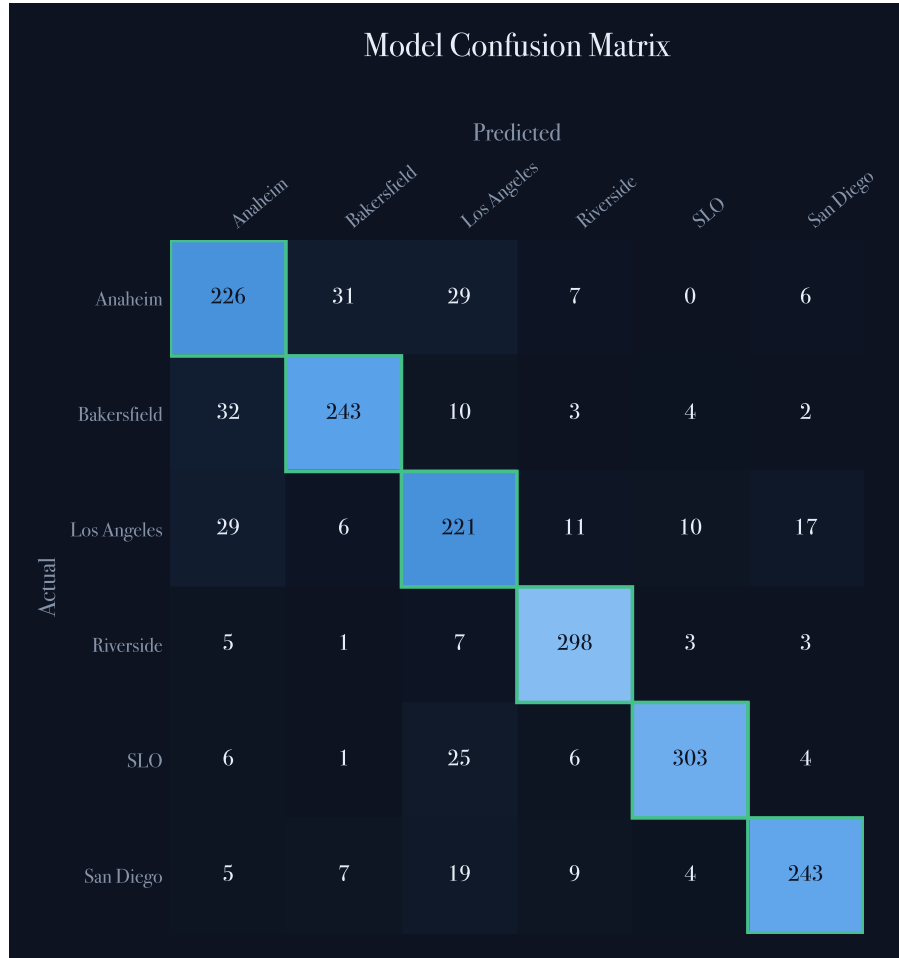


Figure 5: Validation confusion matrix. Rows are the true city and columns the predicted city; diagonal cells, outlined in green, are correct predictions. Color encodes the row-normalized rate, so a bright diagonal indicates strong per-class recall. The errors cluster among the visually similar inland and suburban cities.

References

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, 2012.
- [2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [3] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, 2015.
- [4] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- [5] T. Weyand, I. Kostrikov, and J. Philbin. PlaNet: Photo geolocation with convolutional neural networks. In *European Conference on Computer Vision*, 2016.